

1)What are applications of dbms?

Database Management Systems (DBMS) find applications across various domains and industries due to their ability to efficiently store, retrieve, and manage large volumes of data. Here are some common applications:

1. ***Banking Systems***: DBMS are used to manage customer accounts, transactions, loans, and financial records securely.
2. ***E-commerce Platforms***: Online retailers rely on DBMS to manage product catalogs, customer information, orders, and inventory.
3. ***Healthcare Systems***: Hospitals and clinics utilize DBMS to store and manage patient records, medical histories, treatment plans, and billing information.
4. ***Education Management***: Educational institutions use DBMS for managing student data, course schedules, grades, and academic resources.
5. ***Human Resources***: DBMS help HR departments manage employee information, payroll, benefits, recruitment, and performance evaluations.
6. ***Telecommunication Systems***: Telecom companies use DBMS to store subscriber data, call records, network configurations, and billing information.
7. ***Inventory Management***: Retailers, warehouses, and manufacturers rely on DBMS to track inventory levels, orders, supply chain, and logistics.
8. ***Social Media Platforms***: Social media platforms use DBMS to store user profiles, posts, comments, likes, and other interactions.
9. ***Government Systems***: Government agencies use DBMS for managing citizen data, tax records, land registries, and public services.

10. ***Transportation and Logistics***: Transportation companies use DBMS to track shipments, manage routes, optimize logistics, and monitor vehicle maintenance.

These are just a few examples highlighting the wide-ranging applications of DBMS across industries to streamline operations, enhance decision-making, and improve data management efficiency.

2)File system approach?

In the file system approach to DBMS, data is stored in separate files managed by the operating system. Each file corresponds to a specific application, lacking integration and leading to data redundancy, inconsistency, and difficulties in data access and management. It's a rudimentary method compared to modern DBMS, which offer better data organization, integrity, and security.

3)What is information

In DBMS, information refers to processed and organized data that provides insights and context for decision-making. It's the result of analyzing and presenting raw data in a meaningful format to support understanding and actions by users or systems.

4)What is mean by meta data(data about data)?

Metadata in DBMS refers to data that describes the structure, organization, and characteristics of the stored data. It provides information about data types, relationships, constraints, and other attributes within the database. Essentially, metadata serves as documentation or a roadmap for understanding and utilizing the data effectively within the database system.

5)What is mean by dbms

DBMS stands for Database Management System. It's a software system that facilitates the creation, maintenance, and manipulation of databases. DBMS enables users to store, retrieve, update, and manage data efficiently while ensuring data integrity, security, and concurrency control. It serves as an interface between users and the underlying database, providing tools and utilities for managing data effectively.

6)Three tier architecture

Three-tier architecture in DBMS divides a system into three logical layers:

1. ***Presentation Tier (Client Tier)***: Handles user interface and interaction.
2. ***Application Tier (Middle Tier)***: Manages business logic and processing.
3. ***Data Tier (Backend Tier)***: Stores and manages data.

This architecture enhances scalability, maintainability, and flexibility by separating concerns and functionalities across different layers.

7)What are data models

Data models in DBMS define how data is structured, organized, and related within a database. They provide a conceptual framework for representing real-world scenarios. Common types include:

1. ***Relational Model***: Organizes data into tables with rows and columns.
2. ***Hierarchical Model***: Represents data in a tree-like structure with parent-child relationships.
3. ***Network Model***: Extends the hierarchical model, allowing records to have multiple parent and child relationships.
4. ***Object-Oriented Model***: Represents data as objects with attributes and methods.
5. ***Entity-Relationship Model (ER Model)***: Describes entities, attributes, and relationships between them.

Data models help design and understand databases, ensuring effective data storage, retrieval, and management.

8)Hierarchical model

The hierarchical model in DBMS organizes data in a tree-like structure where each record has a single parent and multiple children. It's based on a one-to-many relationship and resembles a parent-child hierarchy, suitable for representing certain types of data with clear hierarchical relationships. However, it lacks the flexibility of other models like the relational model and may not be ideal for handling complex data structures.

9)How null values are specified in tables

In tables within a DBMS, null values are specified by leaving a field or attribute empty or undefined. This indicates that no value is assigned to that specific field for a particular record. Null signifies the absence of a value rather than a specific value itself, differentiating it from an empty string or zero.

10)What is mean by attribute

In DBMS, an attribute refers to a characteristic or property of an entity within a database. It represents a specific piece of information associated with the entity. Attributes define the structure of the data and

describe the various aspects or qualities of the entities being modeled in the database. They can include fields such as names, dates, numbers, or descriptions, depending on the type of data being stored.

11)Diff between domain and attribute

In DBMS:

- ***Attribute***: Refers to a specific property or characteristic of an entity. It represents a single piece of information associated with the entity. For example, in a "Student" entity, attributes might include "Student ID," "Name," and "Age."

- ***Domain***: Defines the set of possible values that an attribute can have. It specifies the range of valid values for the attribute. For instance, the domain of the "Age" attribute might be defined as integers between 18 and 100.

In short, attributes represent specific properties of entities, while domains define the range of valid values for those attributes.

12)Properties of relational model

The relational model in DBMS is characterized by the following properties:

1. ***Tabular Structure***: Data is organized into tables (relations) composed of rows (tuples) and columns (attributes).
2. ***Uniqueness***: Each row (tuple) in a table is unique, identified by a primary key that ensures no duplicate rows.
3. ***Attribute-based Relationships***: Relationships between tables are established through common attributes, ensuring data integrity and consistency.
4. ***Normalization***: Tables are normalized to minimize redundancy and dependency, reducing data anomalies and ensuring data integrity.
5. ***Structured Query Language (SQL)***: SQL is used to query and manipulate data in relational databases, providing a standardized language for database interactions.

6. ***ACID Properties***: Relational databases adhere to ACID (Atomicity, Consistency, Isolation, Durability) properties to ensure transactional integrity and reliability.

7. ***Data Independence***: The relational model provides both logical and physical data independence, allowing changes to the database schema without affecting application programs.

Overall, the relational model offers a flexible, structured approach to organizing and managing data, facilitating efficient data retrieval, manipulation, and maintenance.

13)What is mean referential integrity and entity integrity

In DBMS:

- ***Referential Integrity***: Refers to the accuracy and consistency of relationships between tables. It ensures that foreign key values in one table correspond to primary key values in another table. This prevents orphaned records and maintains the integrity of the data relationships.

- ***Entity Integrity***: Ensures that each row (tuple) in a table has a unique and non-null primary key value. It guarantees the uniqueness and validity of data within a table, preventing duplicate or null primary key values.

In short, referential integrity maintains consistency in relationships between tables, while entity integrity ensures the uniqueness and validity of data within individual tables.

14)What do you mean by aggregate function

In DBMS, an aggregate function performs a calculation on a set of values and returns a single value as a result. Examples include functions like SUM, AVG, COUNT, MIN, and MAX. These functions are commonly used in SQL queries to summarize or analyze data within a database table or result set.

15)Diff between delete command and drop command

In DBMS:

- ***DELETE Command***: Removes specific rows or records from a table while preserving the table structure. It's used with a WHERE clause to specify the conditions for row deletion.

- ***DROP Command***: Deletes entire tables or objects from the database, including all rows and associated structures. It removes the table's definition and data, essentially eliminating the table from the database schema.

In short, DELETE removes specific rows from a table, while DROP removes entire tables or objects from the database.

16)How to add a new row to a existing query

To add a new row to an existing table in a database using SQL:

1. Use the INSERT INTO statement followed by the table name.
2. Specify the column names within parentheses (optional if you're inserting values into all columns).
3. Use the VALUES keyword followed by the values you want to insert into the corresponding columns.

For example:

```
INSERT INTO TableName (Column1, Column2, ...)  
VALUES (Value1, Value2, ...);
```

This SQL statement adds a new row to the specified table with the provided values for each column.

17)Display all customer names who live on the same street and in the same city and city name is New Delhi

To display all customer names who live on the same street and in the same city (New Delhi), you can use the following SQL query:

```
sql  
  
SELECT customer_name  
FROM customers  
  
WHERE street = 'Street_Name' AND city = 'New Delhi';
```

Replace 'Street_Name' with the actual street name. This query selects customer names from the "customers" table where the street matches the specified street name and the city is 'New Delhi'.

18) Define view

In DBMS, a view is a virtual table generated from the result set of a SELECT query. It does not store data itself but rather presents data from one or more tables in a structured format. Views allow users to query and manipulate data as if it were a regular table, providing a customized perspective of the database schema without altering the underlying data. They offer a way to simplify complex queries, enforce security restrictions, and provide a layer of abstraction over the database schema.

19) Views are virtual tables

Exactly. In DBMS, views are virtual tables that are derived from the result of a SELECT query. They don't store data themselves but provide a way to present a customized perspective of the data stored in one or more underlying tables.

20) What is transaction and how you access transaction support in sql

A transaction in DBMS is a unit of work that is executed within a database management system. It consists of one or more database operations, such as INSERT, UPDATE, DELETE, or SELECT, that are treated as a single logical unit. The ACID properties (Atomicity, Consistency, Isolation, Durability) ensure that transactions are executed reliably.

In SQL, you access transaction support using transaction control statements such as:

- ***BEGIN TRANSACTION***: Begins a new transaction.
- ***COMMIT***: Saves the changes made during the transaction and ends the transaction.
- ***ROLLBACK***: Undoes the changes made during the transaction and ends the transaction.

These statements allow you to manage the execution and outcome of transactions, ensuring data integrity and reliability.

21) States of transaction management

In DBMS, transaction management involves the following states:

1. ***Active***: The transaction is currently being executed and performing database operations.

2. ***Partially Committed***: The transaction has executed all its operations successfully, but the changes have not yet been made permanent in the database.

3. ***Committed***: The transaction has completed successfully, and all its changes have been made permanent in the database.

4. ***Failed***: The transaction encountered an error or exception during execution and cannot be completed.

5. ***Aborted***: The transaction has been rolled back or aborted due to a failure, and any changes made by the transaction have been undone.

These states help maintain the consistency and reliability of the database during transaction processing.

22) How you apply sum function count function (aggregate functions)

In DBMS, you apply aggregate functions like SUM and COUNT to perform calculations on sets of values:

- ***SUM***: Adds up the values in a specified column or expression.

Example: `SELECT SUM(sales_amount) FROM sales;`

- ***COUNT***: Counts the number of rows in a specified column or expression.

Example: `SELECT COUNT(*) FROM customers WHERE age > 30;`

You use these functions in SQL queries to aggregate data and derive useful insights from the database.

23) Write an SQL statement to give every employee a 10% raise

To give every employee a 10% raise in DBMS, you can use an SQL UPDATE statement like this:

SQL

UPDATE employees

SET salary = salary * 1.1;

This statement updates the "salary" column in the "employees" table by multiplying each employee's current salary by 1.1, effectively giving them a 10% raise.

24)What do you mean by assertion

In DBMS, an assertion is a declarative statement that specifies a condition or constraint that must always be true for the database to remain consistent. It acts as a rule or condition that the database must adhere to during data manipulation operations. If an assertion evaluates to false, it indicates a violation of the specified condition, and the DBMS may prevent the operation that caused the violation. Assertions help enforce data integrity and maintain consistency within the database.

25)How do you create assertions

In DBMS, you create assertions using the CREATE ASSERTION statement in SQL. Here's a short example:

```
sql
```

```
CREATE ASSERTION assertion_name
```

```
CHECK (condition);
```

Replace assertion_name with a name for the assertion, and condition with the specific condition or constraint you want to enforce. Assertions are used to ensure data integrity by defining rules that must always hold true in the database.

26)*How do you write triggers

To write triggers in DBMS:

1. Use the CREATE TRIGGER statement in SQL.
2. Specify the trigger name, event (e.g., INSERT, UPDATE, DELETE), and table on which the trigger operates.
3. Define the trigger timing (BEFORE or AFTER the event).
4. Write the trigger action, which can be SQL statements or procedure calls.

For example, to create a trigger that updates a timestamp column whenever a row is inserted or updated in a table:

sql

```
CREATE TRIGGER update_timestamp
AFTER INSERT OR UPDATE ON table_name
FOR EACH ROW
BEGIN
    UPDATE table_name
    SET timestamp_column = CURRENT_TIMESTAMP
    WHERE primary_key_column = NEW.primary_key_column;
END;
```

This trigger updates the timestamp_column with the current timestamp for each new row or update in table_name. Adjust the specifics according to your database schema and requirements.

27)How do you add new customer to the employee table

To add a new customer to the employee table in a DBMS:

Use the SQL INSERT INTO statement:

sql

```
INSERT INTO employee (employee_id, employee_name, department, salary)
VALUES (12345, 'John Doe', 'Sales', 50000);
```

Replace the values with the appropriate data for the new customer, ensuring they match the column order and data types in the employee table.

28)What is the difference between table and view

In short:

- ***Table***: In DBMS, a table is a physical structure that stores data as rows and columns. It holds actual data and supports operations like INSERT, UPDATE, DELETE, and SELECT.

- *View*: A view is a virtual table generated by a SELECT query. It doesn't store data itself but provides a customized perspective of the data stored in one or more tables. Views can simplify complex queries, enforce security, and provide data abstraction.

29)List the name of all students whose name is starting with small 'a'

To list the names of all students whose names start with a lowercase 'a' in a DBMS:

```
sql  
  
SELECT name  
  
FROM students  
  
WHERE name LIKE 'a%';
```

This SQL query retrieves the names of students from the "students" table where the name starts with the letter 'a'. The '%' is a wildcard character that matches any sequence of characters.

30)Add a column city to the student table (ALTER TABLE COMMAND)

To add a column named 'city' to the student table using the ALTER TABLE command:

```
sql  
  
ALTER TABLE student  
  
ADD COLUMN city VARCHAR(50);
```

This SQL statement alters the 'student' table by adding a new column named 'city' with a VARCHAR data type, allowing up to 50 characters. Adjust the data type and size according to your requirements.

31)Find the list of students who have enrolled in computer science

To find the list of students who have enrolled in computer science in a DBMS:

```
sql  
  
SELECT student_name  
  
FROM enrollment
```

WHERE course_name = 'Computer Science';

This SQL query retrieves the names of students from the "enrollment" table where the course name is 'Computer Science'. Adjust the table and column names according to your database schema.

32)Derived attribute

A derived attribute in DBMS is an attribute whose value is derived from other attributes within the database. Unlike regular attributes, the value of a derived attribute is not stored explicitly but is calculated or derived from other attributes through a specified formula or computation. Derived attributes are often used to represent information that can be inferred from existing data, enhancing data organization and analysis without the need for redundant storage.

33)How to maintain referential integrity between two tables

To maintain referential integrity between two tables in DBMS:

1. Use foreign key constraints: Define a foreign key in the child table that references the primary key in the parent table.
2. Ensure that the values in the foreign key column of the child table match the values in the primary key column of the parent table.
3. Specify the appropriate referential actions (e.g., ON DELETE CASCADE, ON UPDATE CASCADE) to enforce actions like cascading deletes or updates when referenced rows are modified.
4. Avoid inserting or updating data that violates the referential integrity constraints.

By enforcing foreign key constraints, the database system ensures that data relationships between the two tables remain consistent and valid.

34)1 tier and 2 tier

- *One-tier architecture*: Also known as standalone architecture, it involves running the entire application on a single machine. Both the user interface and the database reside on the same system. It's simple but lacks scalability and can be challenging to maintain.

- *Two-tier architecture*: Involves separating the application into two parts: the client-side interface for user interaction and the server-side database for data storage and retrieval. It improves scalability and allows for better management of the application logic and data storage.

35)What is mean by physical data independence

And logical data independence

In short:

- ***Physical Data Independence***: Refers to the ability to modify the physical storage structures or organization of data without affecting the application's ability to access or manipulate that data. Changes such as reorganizing storage, adding indexes, or moving to a different storage system can be made without impacting the application.

- ***Logical Data Independence***: Refers to the ability to modify the logical structure or schema of the database without affecting the application's ability to access or manipulate the data. Changes such as adding or modifying tables, views, or relationships can be made without affecting the external schema or programs that use the database.